

1. THE PROBLEM

- Traditional tertiary hostel maintenance relies on **paper logbooks**, verbal reports to porters, and unorganized WhatsApp messages.
- No Audit Trail:** Complaints go missing; zero individual accountability.
 - Location Ambiguity:** Students misreport block/room numbers causing technician delays.
 - SLA Failures:** Urgent faults (electrical, plumbing) linger without escalation.
 - Management Blindspots:** Hall masters lack aggregated metrics on recurring failures.

2. THE HOSTELFIX SOLUTION

- A closed-loop, role-based cloud platform with strict location verification, real-time dispatch, and automated SLA tracking.
- Verified Reporting:** Validated campus hierarchy (Hall > Block > Floor > Room).
 - Role-Based Dispatch:** Automated assignment pipelines for technicians.
 - Proof of Work:** Mandatory photo documentation & student sign-off before closing.
 - Centralized Intelligence:** Real-time MTTR and fault distribution dashboards.

3. TECHNICAL ARCHITECTURE & TECHNOLOGY STACK

[Next.js 14 App Router + Tailwind + Shadcn UI + TanStack Query] —HTTPS / HttpOnly Cookie—> [Node.js + Express + TypeScript ESM] | [MongoDB Atlas (Mongoose ODM)] <—Indexed Queries— [Domain Services / Zod Validation] — | [Redis Cache + BullMQ Background SLA / Worker Queue]

Frontend Stack

- Next.js 14** App Router with server & client components.
- Styling** Tailwind CSS + Shadcn UI (accessible Radix primitives).
- Data Fetching** TanStack Query v5 (optimistic updates, caching).
- Form & Icons** React Hook Form, Zod client validation, Lucide Icons.

Backend & Infrastructure

- Node / Express** Clean modular architecture with strict TypeScript ESM.
- Security** Dual-token JWT (HttpOnly cookies), BCrypt, RBAC policies.
- Database** MongoDB Atlas (compound indexes, Mongoose schemas).
- Async & Media** Redis + BullMQ (SLA monitors), Cloudinary CDN.

4. ROLE-BASED ACCESS CONTROL (5 DEDICATED USER ROLES)

Role	Primary Operational Responsibility	Key Dashboard Action
Student / Resident	Files issues with guided room selectors; uploads fault photos; tracks real-time progress.	Report Issue, Confirm Resolution
Hall Manager	Oversees assigned hall; reviews, prioritizes, and assigns work orders to technicians.	Triage & Assign Technicians
Maintenance Tech	Receives work orders; updates progress (Assigned > In Progress > Resolved) with work notes.	Update Status & Upload Proof
University Admin	Campus-wide oversight; compares cross-hall MTTR, vendor costs, and recurring fault hotspots.	Campus Analytics & Reports
System Admin	Manages student/staff accounts, configures hall/block structures, inspects system audit logs.	User Provisioning & Audit Logs

FIRST 2-MINUTE DEFENCE QUESTIONS

Strategic Responses for Panel Examination & Technical Vetting

RAPID
ANSWERS

Key Strategy: Be crisp, confident & academic.
State *problem* > *decision* > *metric/impact*.

Q1: "Why build another maintenance system? What is novel here over WhatsApp or Google Forms?"

Problem & Novelty

Strategic Answer: "Ad-hoc tools like WhatsApp and Google Forms are unstructured and open-ended. They fail because they cannot enforce **room-level location verification**, cannot track **Mean Time to Resolution (MTTR)**, and have no **automated SLA escalation** when water leaks or electrical faults remain unresolved. HostelFix introduces closed-loop governance: tickets are bound to verified residential entities, managed via strict role-based state machines, audited at every step, and quantified for administrative accountability."

Q2: "Why choose MongoDB (NoSQL) over a Relational Database like PostgreSQL?"

Database Decision

Strategic Answer: "University hostels feature diverse, non-uniform architectural typologies: some halls have standard Block-Floor-Room layouts, while others have Annexes, chalets, or shared communal blocks. MongoDB's document model naturally accommodates this flexible location hierarchy and polymorphic issue attachments without requiring complex multi-table joins. Crucially, we enforce strict referential consistency at the application layer through **Zod schemas, Mongoose population, and compound indexes** on foreign references such as `assignedHallIds` and `allocatedLocationId`."

Q3: "How is security and data isolation maintained across different halls of residence?"

Security & RBAC

Strategic Answer: "We implement defense-in-depth across authentication and authorization. For authentication, we use dual-token JWTs stored in **HTTP-only, SameSite secure cookies**, effectively mitigating XSS and CSRF token interception. For authorization, our backend enforces policy guards at the service layer: a Hall Manager or Maintenance Technician can only query and mutate records scoped to their explicitly assigned hall IDs. Sensitive student contact information is redacted, and every critical state mutation generates an immutable audit log entry."

Q4: "How does the system handle high concurrency during move-in week when hundreds report faults?"

Scalability & Load

Strategic Answer: "We decoupled our workload across three key architectural tiers: First, all media uploads stream directly to **Cloudinary CDN**, keeping the Node.js event loop unblocked. Second, non-blocking asynchronous tasks (email alerts, SLA monitors, audit archiving) are offloaded to an in-memory **Redis queue powered by BullMQ**. Third, database queries utilize indexed projections and lean queries, while the frontend leverages **TanStack Query client-side caching** with optimistic updates to minimize duplicate network traffic."

Q5: "What was your greatest technical hurdle, and how did you resolve it?"

Technical Depth

Strategic Answer: "The primary challenge was designing a robust, multi-tenant location model that cleanly supports granular room allocations while maintaining sub-100ms response times for cross-hall queries. We solved this by structuring a normalized location graph populated selectively during authentication, combined with caching on the client. Additionally, when technicians experienced missing hall crest data due to shallow auth tokens, we implemented deep population within the auth refresh pipeline, ensuring seamless state hydration across all 5 user roles."

Key Takeaways for Your 2-Minute Opening Pitch:

Pitch: "HostelFix transitions university residential management from reactive, pen-and-paper guesswork to a proactive, data-driven maintenance lifecycle. Built with Next.js 14 and Node.js, it unites students, technicians, and administrators into a single audited platform that reduces ticket resolution times and preserves campus facilities."